[강추] matplotlib를 이용하여 기본 그래프를 그리기

https://bcho.tistory.com/1201

1. plt nujmpy 이용한 그래프 만들기
2. Data를 어레이로 읽어들여 사용
3. 크기 조절 등 엄청 좋구나
4. 고맙습니다. 조대현임

파이썬 이용한 데이타 시각화 #1 - Matplotlib 기본 그래프 그리기
조대협 (http://bcho.tistory.com)

백앤드 엔지니어와 백그라운드를 가진 경험상, 머신러닝을 공부하면서 헷갈렸던 부분중 하나가, 데이타에 대한 시각화이다. 머신러닝은 모델을 구현하는 것도 중요하지만, 학습할 데이타를 선별하고, 만들어진 모델을 검증하기 위해서는 데이타를 이해하는 것이 필수적이고 이를 위해서는 데이타를 시각화 해서 보는 것이 매우 중요하다.

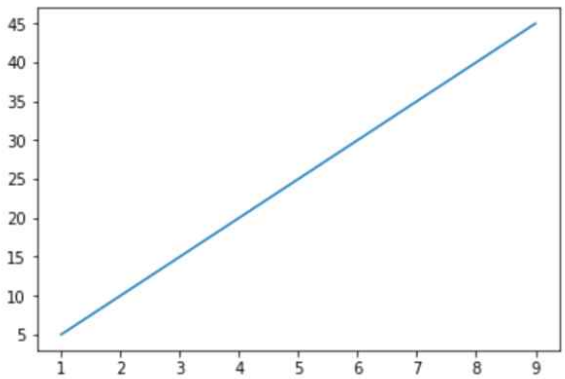
그동안 그래프를 그리는 것을 스택오버플로우등에서 찾아서 복붙을 해서 사용하다가 matplotlib를 정리해야겠다고 해서 메뉴얼을 봤지만 도무지 이해가 되지 않아서, 결국 온라인 강좌를 들어서 정리해봤는데, 역시 강좌를 들으니까는 훨씬 빠르게 이해가 된다. 참고한 코스는 datacamp에 있는 “” 코스이다.

오늘은 matplotlib를 이용하여 기본적인 그래프를 그리는 방법에 대해서 정리하도록 한다.

1. 기본 그래프 그리기

기본적인 그래프를 그리기 위해서는 matplotlib.pyplot에서 plot(x,y)를 사용하면 된다. x,y는 각각 X축과 Y축의 값이 된다.

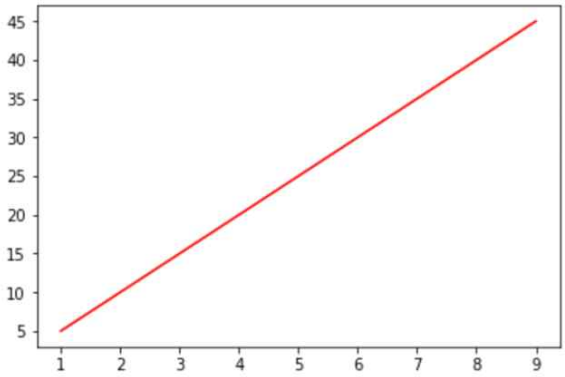
```
from matplotlib import pyplot as plt
import numpy as np

x = np.arange(1,10)
y = x*5

plt.plot(x,y)
plt.show()
```



2. 색깔 바꾸기

그래프를 그릴때 선의 색을 지정하기 위해서는 plot에서 인자로 컬러를 주면된다. 컬러표는 아래를 참고하면 되고 붉은색은 r, 파란색은 b으로 정의한다.

```
from matplotlib import pyplot as plt
import numpy as np

x = np.arange(1,10)
y = x*5

plt.plot(x,y,'r')
plt.show()
```



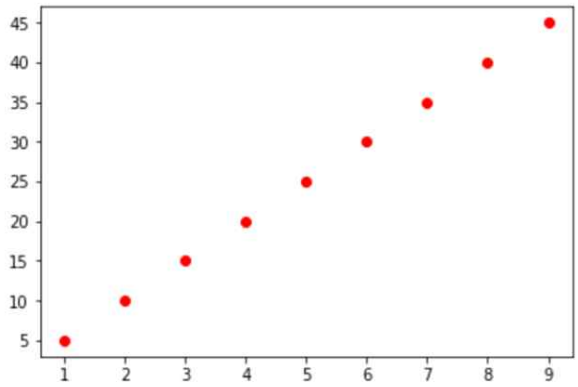
Character	Color
'b'	Blue
'g'	Green
'r'	Red
'c'	Cyan
'm'	Magenta
'y'	Yellow
'k'	Black
'w'	White

3. 선 종류 변경하기

선을 그릴때, 다양한 선의 종류를 선택할 수 있다. 디폴트가 직선이고, 점으로 표현하는 마커나 점선등을 선택할 수 있다. 선의 선택은 plot에서 세번째 인자에 선의 종류를 지정하면 되고, 색을 같이 지정하려면 다음문자에 색을 지정하면 된다 다음은 동그란 마커 'o'를 붉은색 'r'로 표현하기 때문에, 세번째 인자를 'or'로 전달하였다.

```
from matplotlib import pyplot as plt
import numpy as np
```

```
x = np.arange(1,10)
y = x*5

plt.plot(x,y,'or')
plt.show()
```



다음은 선에 대한 종류표이다.

4. 라벨과 타이틀

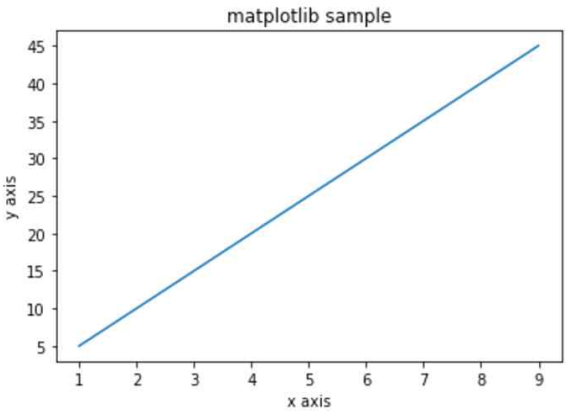
그래프를 그릴때 그래프의 타이틀과 X,Y축의 라벨을 표현하기 위해서는 타이틀은 plt.title("타이틀명"), X,Y축에 대한 라벨은 plt.xlabel('X축 라벨명'), plt.ylabel('Y축 라벨명') 을 사용한다.

```
from matplotlib import pyplot as plt
import numpy as np

x = np.arange(1,10)
```

Character	Description
'-'	Solid line style
'--'	Dashed line style
'-.'	Dash-dot line style
':'	Dotted line style
'.'	Point marker
','	Pixel marker
'o'	Circle marker
'v'	Triangle_down marker
'^'	Triangle_up marker
'<'	Triangle_left marker
'>'	Triangle_right marker
'1'	Tri_down marker
'2'	Tri_up marker
'3'	Tri_left marker
'4'	Tri_right marker
's'	Square marker
'p'	Pentagon marker
'*'	Star marker
'h'	Hexagon1 marker
'H'	Hexagon2 marker
'+'	Plus marker
'x'	X marker
'D'	Diamond marker
'd'	Thin_diamond marker
'|'	Vline marker
'_'	Hline marker

```
y = x*5

plt.plot(x,y)
plt.xlabel('x axis')
plt.ylabel('y axis')
```

```
plt.title('matplotlib sample')
plt.show()
```

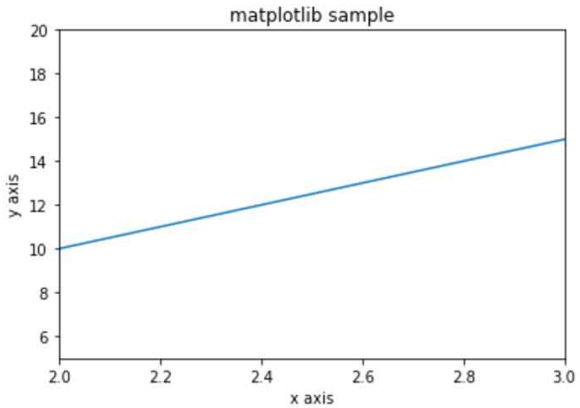


5. 구간 확대/축소

그래프는 입력되는 x,y의 최소,최대 구간으로 자동으로 그려지는데, 이 구간을 키우거나 줄이기 위해서 x,y의 구간을 정의할 수 있다. x축은 plt.xlim(최소,최대), y축은 plt.ylim(최소,최대)로 정의하면 된다.
아래는 x축을 2~3, y축을 5~20으로 확대해서 그래프를 그리는 예제이다.

```
from matplotlib import pyplot as plt
import numpy as np

x = np.arange(1,10)
y = x*5

plt.xlim(2,3)
plt.ylim(5,20)
plt.plot(x,y)
plt.xlabel('x axis')
plt.ylabel('y axis')
```

```
plt.title('matplotlib sample')
plt.show()
```

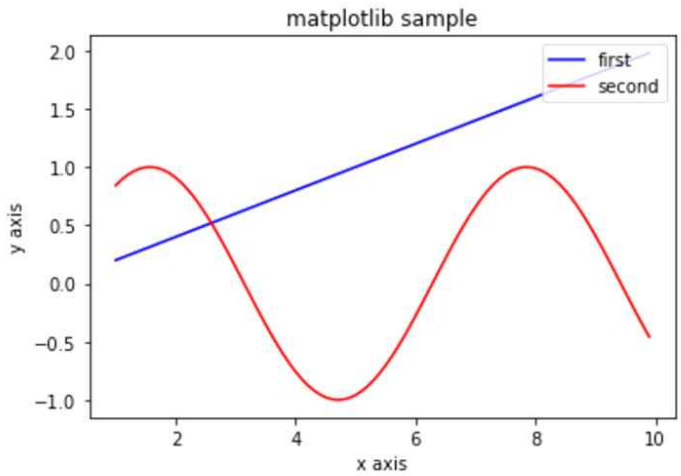


6. 레전드

그래프를 그릴때 여러개의 그래프를 같이 그릴 수 있는데, 이경우 각 그래프가 구분이 안되기 때문에, 그래프마다 라벨을 달고 이 라벨명을 출력할 수 있는데, 이를 legend라고 한다.
아래는 first와 second 라는 두개의 그래프를 그리고, 우측 상단에 legend를 표현한 예이다.
legend를 사용하기 위해서는 plt.plot에서 label 변수에 그래프의 이름을 정의하고, plt.legend('위치')를 정해주면 legend를 그래프상에 표현해주는데, legend의 위치는 아래 표를 참고하면 된다.

```
from matplotlib import pyplot as plt
import numpy as np

x = np.arange(1,10,0.1)
y = x*0.2
y2 = np.sin(x)
```

```
plt.plot(x,y,'b',label='first')
plt.plot(x,y2,'r',label='second')
plt.xlabel('x axis')
plt.ylabel('y axis')
plt.title('matplotlib sample')
plt.legend(loc='upper right')
plt.show()
```



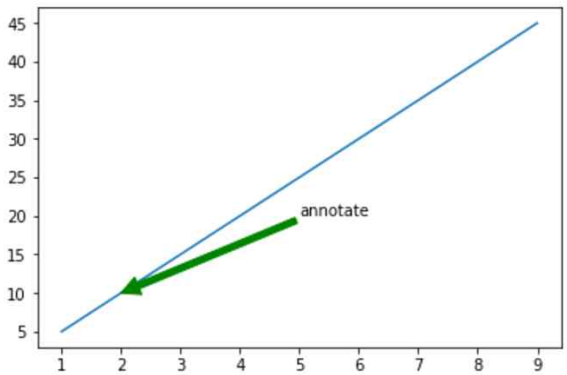
string	code	string	code	string	code
'upper left'	2	'upper center'	9	'upper right'	1
'center left'	6	'center'	10	'center right'	7
'lower left'	3	'lower center'	8	'lower right'	4
'best'	0			'right'	5

7. 어노테이션

다음은 어노테이션이라는 기능으로, 그래프에 화살표를 그린후, 그 화살표에 문자열을 출력하는 기능이다. 예를들어 “이값이 최소값" 이런식으로 화살표를 그려서 표현할때 사용하는데 plt.annotate 함수를 사용하면 된다.

plt.annotate(‘문자열',xy,xytext,arrowprops) 식으로 사용한다.
문자열은 어노테이션에서 나타낼 문자열이고, xy는 화살표가 가르키는 점의 위치, xytext는 문자열이 출력될 위치, arrowprops는 화살표의 속성으로 칼라등을 정의한다.

```
from matplotlib import pyplot as plt
import numpy as np

x = np.arange(1,10)
y = x*5

plt.plot(x,y)
plt.annotate('annotate',xy=(2,10),xytext=(5,20),arrowprops={'color':'green'})
plt.show()
```



8. 서브플롯

여러개의 그래프를 그리고 싶을때가 있는데, 이 경우 서브플롯이라는 것을 사용한다. 서브플롯은 그래프가 그려질 위치를 격자형으로 지정하는데, plt.subplot(nrow,ncol,pos) 식으로 사용한다.
nrow,ncol은 그래프를 그린 plain의 크기를 지정하는데, 3,2면

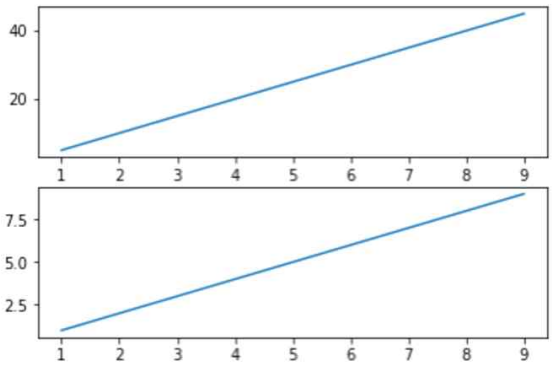
3줄로, 가로는 2칸으로 된 그래프 plain 설정한다. 그리고 마자막 pos는 몇번째 plain에 그래프를 그릴지 지정하는데, 아래와 같이 상단에서 부터 우측,아래 방향으로 1,2,3,4,5,6 순서가 된다.

1	2
3	4
5	6

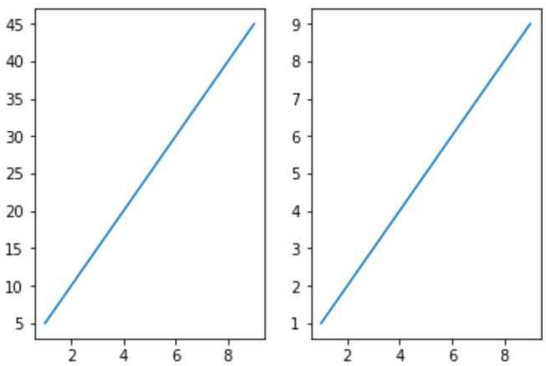
아래 그림은 2,1 크기의 plain 을 만들어놓고 그래프를 위,아래로 두개를 그리는 예제이다.

```
from matplotlib import pyplot as plt
import numpy as np

x = np.arange(1,10)
y1 = x*5
y2 = x*1
y3 = x*0.3
y4 = x*0.2

plt.subplot(2,1,1)
plt.plot(x,y1)
plt.subplot(2,1,2)
plt.plot(x,y2)
plt.show()
```

아래 그림은 한줄의 두칸 plain을 만들어놓고, 좌우에 두개의 그래프를 그린 예제이다.

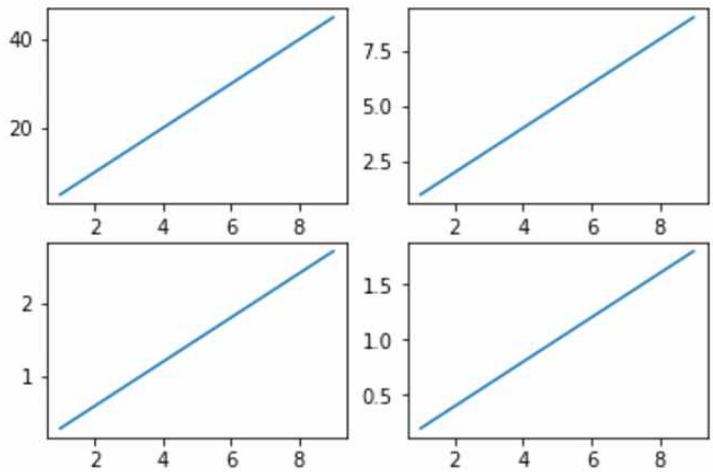



```
from matplotlib import pyplot as plt
import numpy as np

x = np.arange(1,10)
y1 = x*5
y2 = x*1
y3 = x*0.3
y4 = x*0.2

plt.subplot(1,2,1)
plt.plot(x,y1)
plt.subplot(1,2,2)
plt.plot(x,y2)
plt.show()
```



다음은 2x2 plain으로 4개의 그래프를 그린 예제이다.

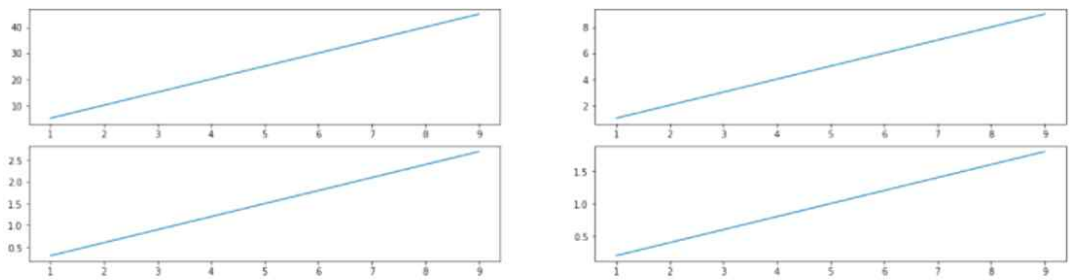
```
from matplotlib import pyplot as plt
import numpy as np

x = np.arange(1,10)
y1 = x*5
y2 = x*1
y3 = x*0.3
y4 = x*0.2

plt.subplot(2,2,1)
plt.plot(x,y1)
plt.subplot(2,2,2)
plt.plot(x,y2)
plt.subplot(2,2,3)
plt.plot(x,y3)
plt.subplot(2,2,4)
plt.plot(x,y4)
plt.show()
```



9. 그래프 사이즈

그래프를 크게 그리고 싶을때 그래프 자체의 크기를 변경할 수 있는데, plt.figure를 이용하여 figsize=(가로,세로)를 인자로 주면 그래프가 그려질 전체 그림의 크기를 조절할 수 있다. 아래는 20x5 크기로 그래프를 그릴 크기를 지정하는 예제이다.

```
import numpy as np

x = np.arange(1,10)
y1 = x*5
y2 = x*1
y3 = x*0.3
y4 = x*0.2

plt.figure(figsize=(20,5))
plt.subplot(2,2,1)
plt.plot(x,y1)
plt.subplot(2,2,2)
plt.plot(x,y2)
plt.subplot(2,2,3)
plt.plot(x,y3)
plt.subplot(2,2,4)
plt.plot(x,y4)
plt.show()
```



지금까지 간단하게 matplotlib를 이용하여 기본 그래프를 그리는

방법에 대해서 알아보았다. 다음글은 바차트,히스토그램등 다양한 그래프 타입에 대해서 알아본다.